



Informatikai és Villamosmérnöki Kar

Valós idejű Polip Szegmentáló Keretrendszer

Szakdolgozat

Belső konzulens:

Prof. Dr. Horváth Zoltán

Külső konzulens:

—

Készítette:

Birkás Levente

Tartalomjegyzék

1	Bevezetés	4
1.1	Motiváció és klinikai háttér	4
1.1.1	A kolonoszkópia szerepe és az „elmulasztott polipok” problémája	4
1.1.2	A vastagbélrák népegészségügyi jelentősége	4
1.1.3	A számítógépes diagnosztika szükségessége a valós idejű vizsgálatok során	5
1.2	Technológiai kihívások	5
1.2.1	Élő kép megjelenítés és igény szerinti elemzés követelményei	5
1.2.2	HDMI videójel feldolgozásának nehézségei orvosi környezetben	5
1.2.3	Különböző AI modellek integrációja	6
1.3	A szakdolgozat célkitűzései	6
1.3.1	AI-alapú szegmentáló modell fejlesztése és tanítása	6
1.3.2	Moduláris Python alkalmazás élő videó megjelenítésre és gombnyomásra történő szegmentálásra és klasszifikációra	6
1.3.3	Interfész biztosítása külső osztályozó modulok számára	7
1.4	A dolgozat felépítése	7
2	Elméleti háttér és módszertan	8
2.1	Alapfogalmak és definíciók	8
2.1.1	Szemantikus szegmentáció vs. objektum detektálás	8
2.1.2	Kiértékelési metrikák	8
2.1.3	Valós idejű rendszerek teljesítménymutatói	9
2.2	Irodalmi áttekintés	10
2.2.1	Mélytanulás az orvosi képfeldolgozásban	10
2.2.2	A DeepLab modellcsalád és az ASPP modul	10
2.2.3	Valós idejű videófeldolgozás Python környezetben	10
2.3	Alkalmazott eszközök és technológiák	11
2.3.1	Programozási környezet	11
2.3.2	Képfeldolgozó könyvtárak	11
2.3.3	Hardveres környezet	11
3	A szegmentáló modell tervezése és implementációja	13
3.1	Adatbázisok és előkészítés	13
3.1.1	Adatbázisok bemutatása	13
3.1.2	Adattisztítás és annotációs kihívások	13
3.1.3	Augmentációs stratégiák	14
3.2	Modell architektúra	14

3.2.1	DeepLabV3+ kiválasztásának indoklása	14
3.2.2	MobileNetV2 backbone előnyei	15
3.2.3	Veszteségfüggvény és optimalizáció	15
3.3	Tanítási és validációs stratégia	16
3.3.1	Transzfer tanulás	16
3.3.2	A StratifiedGroupKFold jelentősége	16
3.3.3	Hiperparaméterek	16
4	A szoftveralkalmazás tervezése és fejlesztése	18
4.1	Rendszerarchitektúra és követelmények	18
4.1.1	Funkcionális követelmények	18
4.1.2	Nem-funkcionális követelmények	18
4.1.3	Heterogén keretrendszerek integrációja	19
4.1.4	Adatfolyam diagram	19
4.2	Implementációs részletek	20
4.2.1	Videó rögzítő modul	20
4.2.2	Inferáló motor	20
4.2.3	Konfidencia-alapú döntéstámogatás és a klasszifikátor interfész	20
4.3	Felhasználói felület terve	21
4.3.1	Orvosi felhasználói élmény	21
4.3.2	Vizualizációs megoldások	21
4.3.3	Adatgyűjtés és automatikus mentés	22
5	Eredmények és kiértékelés	23
5.1	A szegmentáló modell teljesítménye	23
5.1.1	Kvantitatív eredmények	23
5.1.2	Kvalitatív elemzés	24
5.2	Rendszerteljesítmény mérése	24
5.2.1	Megjelenítési sebesség és igény szerinti elemzési idő	24
5.2.2	Teljes késleltetés és erőforrás-használat	25
5.3	Diszkusszió	25
5.3.1	A gombnyomásra történő elemzés korlátai	25
5.3.2	Illeszthetőség a munkafolyamatba	25
6	Összefoglalás és jövőbeli tervek	27
6.1	Az elért eredmények összefoglalása	27
6.2	A fejlesztés korlátai	27
6.3	Jövőbeli fejlesztési irányok	28
6.3.1	Hardveres optimalizáció	28
6.3.2	Temporális utófeldolgozás	28
6.3.3	További integráció	28
A	A kolorektális polip szegmentáló modell szoftveres fejlesztésének dokumentációja	30
A.1	Cél és kontextus	30
A.1.1	A dokumentáció célja	30
A.1.2	Kapcsolat a dolgozattal	30
A.1.3	Felhasználási szcenárió	30
A.2	Fejlesztési környezet és függőségek	31

A.2.1	Hardverkörnyezet	31
A.2.2	Szoftverkörnyezet és verziók	31
A.2.3	Környezet felállítása	31
A.3	Rendszerarchitektúra magas szinten	31
A.4	Adatkezelés és előfeldolgozás	32
A.4.1	Címtárszerkezet	32
A.4.2	PyTorch Dataset és Dataloader	32
A.4.3	Transzformációk	33
A.5	Modellkomponens	33
A.5.1	Inicializálás és architektúra	33
A.5.2	Veszteségfüggvény és Optimalizáló	33
A.6	Tanítási és validációs folyamat	34
A.6.1	Tanító ciklus	34
A.6.2	Validáció és korai megállás	34
A.7	Eredmények reprodukálhatósága	34
A.7.1	Seed beállítások	34
A.7.2	Hiperparaméter konfiguráció	35
A.8	Kimenetek és használat	35
A.8.1	Checkpoint kezelés	35
A.8.2	Predikciós pipeline kód minta	35
A.9	Ismert limitációk és jövőbeli bővítés	36
B	A valós idejű szegmentáló alkalmazás szoftveres dokumentációja	37
B.1	Cél és kontextus	37
B.1.1	A dokumentáció célja	37
B.1.2	Kapcsolat a dolgozattal	37
B.1.3	Felhasználási szcenárió	37
B.2	Fejlesztési környezet és függőségek	38
B.2.1	Hardverkörnyezet	38
B.2.2	Szoftverkörnyezet és verziók	38
B.2.3	Környezet felállítása és futtatás	38
B.3	Rendszerarchitektúra magas szinten	38
B.4	Modulok részletesen	39
B.4.1	VideoCaptureThread	39
B.4.2	MedicalAnalyzer	39
B.4.3	App (GUI)	40
B.5	Konfiguráció és fájlstruktúra	40
B.5.1	Konfigurációs változók	40
B.5.2	Várt fájlstruktúra	40
B.6	Adatfolyam és szálkezelés	41
B.6.1	Szálmodell	41
B.6.2	Elemzési folyamat (Capture & Analyze)	41
B.7	Ismert limitációk és jövőbeli bővítés	41
B.8	Reprodukálhatóság és futtatási lépések	41

fejezet 1

Bevezetés

Becsült terjedelem: 2-3 oldal

1.1 Motiváció és klinikai háttér

1.1.1 A kolonoszkópia szerepe és az „elmulasztott polipok” problémája

A vastagbélrák szűrésének és megelőzésének jelenlegi „gold standard” eljárása a kolonoszkópia, amely lehetővé teszi a vastagbél nyálkahártyájának közvetlen vizuális ellenőrzését, valamint a prekursor léziók, azaz a polipok azonnali eltávolítását (polypectomia). Bár ez a vizsgálat a leghatékonyabb eszköz a gasztroenterológusok kezében, a diagnosztikai pontosságot erősen befolyásolják az emberi tényezők. Az endoszkópos orvos fáradtsága, a figyelem lankadása, vagy a vizsgálat technikai nehézségei (pl. rossz bél-előkészítés) jelentős hibarányhoz vezethetnek. A szakirodalmi adatok szerint a polipok elmulasztási aránya (miss rate) elérheti a 20-25%-ot is [1], különösen a kisméretű, lapos (sessile) vagy a bélredők mögött megbújó elváltozások esetében. Minden egyes fel nem ismert adenoma növeli a későbbi intervallum-rák kialakulásának kockázatát, ezért az Adenoma Detektálási Arány (ADR) növelése klinikai prioritás.

1.1.2 A vastagbélrák népegészségügyi jelentősége

A colorectalis rák (CRC) napjaink egyik legjelentősebb népegészségügyi kihívása világszerte. A WHO és a globális rákkutatási szervezetek (GLOBOCAN) statisztikái szerint a vastagbélrák a harmadik leggyakoribb diagnosztizált ráktípus, és a második vezető halálok a daganatos megbetegedések körében [2]. A betegség patogenezeise jellemzően a jóindulatú polipokból indul ki, amelyek évek alatt rosszindulatúvá fajulhatnak. Ez a lassú progresszió lehetőséget ad a hatékony prevencióra: a polipok időben történő felismerése és eltávolítása megszakítja a rákosodás folyamatát, így bizonyítottan csökkenti mind az incidenciát, mind

a mortalitást. Ebben a kontextusban a mesterséges intelligencia alapú döntéstámogató rendszerek nem csupán technológiai újítást, hanem életmentő eszközt jelenthetnek.

1.1.3 A számítógépes diagnosztika szükségessége a valós idejű vizsgálatok során

A diagnosztikai hibák csökkentése és az adenoma detektálási arány (ADR) növelése érdekében egyre nagyobb igény mutatkozik olyan számítógépes diagnosztikai (CADx) rendszerek iránt, amelyek valós időben képesek támogatni az endoszkópos szakemberek munkáját. Egy ilyen mesterséges intelligencia alapú "második szempár" képes valós időben feldolgozni a videófolyamot, vagy igény szerint (on-demand) azonnali elemzést nyújtani a gyanús területekről. A rendszer architektúrája biztosítja az alacsony késleltetésű (low-latency) inferenciát, amely elengedhetetlen a klinikai munkafolyamatba való illesztéshez, ezzel minimalizálva az emberi mulasztás kockázatát. A CAD rendszerek legfőbb előnye, hogy mentesek a biológiai fáradtságtól, és objektív, konzisztens döntéstámogatást nyújtanak még a hosszú műszakok végén is. Ezen felül csökkentik a vizsgálok közötti szubjektív eltéréseket (inter-observer variability), és különösen hasznosak lehetnek a kevésbé tapasztalt rezidensek képzésében, mivel vizuálisan ráirányítják a figyelmet a nehezen észrevehető elváltozásokra.

1.2 Technológiai kihívások

1.2.1 Élő kép megjelenítés és igény szerinti elemzés követelményei

A klinikai környezetben a rendszer két elkülönülő követelményt teljesít: az élő videó folyamatos, zökkenőmentes megjelenítését és a felhasználó által kezdeményezett, azonnali elemzést. Az élő megjelenítésnél a megjelenítési sebességnek (FPS) el kell érnie a vizsgálat során kényelmes érzetet nyújtó szintet (általában 25–30 képkocka/másodperc), hogy az orvos ne érezze a késleltetést vagy a megakadást. Az elemzés nem minden képkockán történik, hanem gombnyomásra (vagy rövidített billentyűre): ekkor a rendszer a pillanatnyi képkockát rögzíti, szegmentálja és osztályozza. Ennél a működésmódnál a fő teljesítménymutató a válaszidő (end-to-end latency), azaz a gombnyomás és a képernyőn megjelenő szegmentált eredmény és klasszifikáció közötti idő, amelynek célszerűen 100-200 milliszekundum alatt kell maradnia, hogy a felhasználói élmény folyamatos maradjon.

1.2.2 HDMI videójel feldolgozásának nehézségei orvosi környezetben

Az orvosi endoszkópos tornyokból érkező videójel digitális feldolgozása számos technikai akadályba ütközik, kezdve a megfelelő capture hardver kiválasztásától a jelfolyam stabil, veszteségmentes beolvasásáig. A nagyfelbontású (Full HD vagy 4K) HDMI jelek valós idejű digitalizálása és a képkockák szinkronizálása a feldolgozó algoritmussal kritikus fontosságú,

mivel bármilyen képtörés vagy csúszás diagnosztikai bizonytalanságot okozhat. További nehézséget jelent a különböző gyártók (Olympus, Pentax, Fujifilm) eltérő színprofiljainak és videóformátumainak kezelése, valamint a nagy sávszélességű adatfolyam memóriába történő gyors mozgatása (Direct Memory Access - DMA) anélkül, hogy a CPU-t túlterhelnénk, ami a feldolgozás lassulásához vezetne.

1.2.3 Különböző AI modellek integrációja

Komoly szoftverarchitekturális kihívást jelent több, eltérő feladatot ellátó mélytanuló modell – például egy erőforrás-igényes szemantikus szegmentáló és egy klasszifikációs hálózat – párhuzamos futtatása egyetlen keretrendszeren belül. A Python környezet sajátosságaiból adódóan a modellek közötti hatékony erőforrás-megosztás és az adatátvitel optimalizálása elengedhetetlen a rendszer összeomlásának vagy belassulásának elkerülése érdekében. A GPU memória (VRAM) korlátos erőforrás, amelyért a különböző modellek versengenek; emellett a Python Global Interpreter Lock (GIL) mechanizmusa megnehezíti a valódi párhuzamosítást (multithreading), ezért gyakran multiprocessing megoldásokra vagy aszinkron végrehajtásra van szükség a zökkenőmentes felhasználói élmény biztosításához.

1.3 A szakdolgozat célkitűzései

1.3.1 AI-alapú szegmentáló modell fejlesztése és tanítása

A dolgozat elsődleges célja egy olyan nagy pontosságú, DeepLabV3+ architektúrára épülő konvolúciós neurális hálózat implementálása és tanítása, amely képes a vastagbél-polipok pixel-szintű szegmentálására. A modell fejlesztése során kiemelt figyelmet fordítunk a MobileNetV2 backbone alkalmazására, hogy megtaláljuk az optimális egyensúlyt a detektálási pontosság és a futási sebesség között, minimalizálva a számítási költséget a pontosság jelentős csökkenése nélkül.

1.3.2 Moduláris Python alkalmazás élő videó megjelenítésre és gombnyomásra történő szegmentálásra és klasszifikációra

A második célkitűzés egy moduláris Python alkalmazás megvalósítása, amely élő videóforrásból (kamera, videófájl vagy szimulációs képmappa) folyamatosan megjeleníti a képet, és a felhasználó gombnyomására (“Capture & Analyze”) a pillanatnyi képkockán elvégzi a polip szegmentálást és a klasszifikációt. A szoftver “hídként” funkcionál a bemeneti eszköz és a mélytanulási modellek között: a videófolyamot dedikált szál kezeli, míg az elemzés igény szerint, a rögzített képkockán fut. A megjelenítés három panelből áll: élő kép, a hálózatnak beadott (középre vágott) bemenet és a szegmentált polip eredmény; a klasszifikátor kimenete (pl. adenóma/hiperplastikus) szintén megjelenik. Az architektúra lehetővé teszi külső

osztályozó modulok cseréjét a meglévő interfész révén.

1.3.3 Interfész biztosítása külső osztályozó modulok számára

A rendszer jövőtállósága és bővíthetősége érdekében célkitűzésként szerepelt egy szabványosított interfész (API) kidolgozása, amely lehetővé teszi külső, akár más fejlesztők által készített osztályozó (klasszifikációs) modulok egyszerű integrációját. Ez a “plug-and-play” architektúra biztosítja, hogy a szegmentált polipokról készült képrészleteket (ROI) a rendszer automatikusan továbbíthassa szövettani becslést végző algoritmusoknak, így támogatva a komplex diagnosztikai döntéshozatalt.

1.4 A dolgozat felépítése

A szakdolgozat szerkezeti felépítése a logikai fejlesztési folyamatot követi: a bevezetést követően a második fejezetben ismertetem a szükséges elméleti háttérrel és a felhasznált technológiákat. A harmadik fejezet a szegmentáló modell tervezését és tanítását, míg a negyedik a szoftveralkalmazás implementációját részletezi, végül az ötödik fejezetben bemutatom az elért eredményeket és a rendszer teljesítményének kiértékelését. A hatodik fejezet összefoglalja a kutatás eredményeit és kijelöli a lehetséges jövőbeli fejlesztési irányokat.

fejezet 2

Elméleti háttér és módszertan

Becsült terjedelem: 8-10 oldal

2.1 Alapfogalmak és definíciók

2.1.1 Szemantikus szegmentáció vs. objektum detektálás

A számítógépes látás területén alapvető különbséget kell tenni az objektum detektálás (object detection), amely téglalap alakú keretekkel (bounding box) jelöli a tárgyakat, és a szemantikus szegmentáció között, amely pixel-szintű besorolást végez. Jelen dolgozatban a szemantikus szegmentáció mellett szóló érvek kerülnek előtérbe, mivel orvosi szempontból a polipok pontos alakjának és kiterjedésének meghatározása kritikusabb információt hordoz, mint pusztán a pozíciójuk behatárolása. A pixelpontos maszk lehetővé teszi a lézió területének kvantitatív mérését (pl. térfogat- vagy felszínbecslés), a reszekciós szegélyek pontosabb megítélését, valamint a vizsgálatok egységes, reprodukálható dokumentálását. A klinikai gyakorlatban a szemantikus szegmentáció vizuális kiemelés formájában jelenik meg: a képernyőn a gyanús terület színes maszkja azonnal felhívja az orvos figyelmét a potenciális elváltozásra, csökkentve a kognitív terhelést és a figyelmi hibák kockázatát. Emellett a maszkok hosszú távon oktatási és audit célokat is szolgálhatnak, mivel lehetővé teszik a vizsgálatok utólagos elemzését és a detektálási teljesítmény objektív összehasonlítását.

2.1.2 Kiértékelési metrikák

A szemantikus szegmentációs modellek teljesítményének objektív összehasonlításához elengedhetetlen a szakirodalomban elfogadott matematikai metrikák definiálása. A dolgozat során két alapvető mérőszámot, az Intersection over Union (IoU) és a Dice-együtthatót (Dice Similarity Coefficient) alkalmazzuk a prediktált maszk (P) és a valós referencia maszk (G , Ground Truth) közötti hasonlóság mérésére.

Metszet és unió hányadosa

Az IoU, más néven Jaccard-index, a legelterjedtebb metrika a szegmentációs feladatokban. Értéke a predikció és a valós maszk metszetének és uniójának hányadosa:

$$IoU(P, G) = \frac{|P \cap G|}{|P \cup G|} = \frac{|P \cap G|}{|P| + |G| - |P \cap G|} \quad (2.1)$$

Az IoU értéke 0 és 1 (vagy 0% és 100%) között változik; a 0 az átfedés teljes hiányát, az 1 a tökéletes egyezést jelenti. Mivel az unió (a nevező) magában foglalja a hibásan detektált területeket (fals pozitív) és a nem detektált területeket (fals negatív) is, az IoU szigorúan bünteti a hibákat.

Dice-együttható

A Dice-együttható, amely matematikailag ekvivalens az F1-score-ral, a metszet kétszeresét viszonyítja a két halmaz elemszámának összegéhez:

$$Dice(P, G) = \frac{2 \cdot |P \cap G|}{|P| + |G|} \quad (2.2)$$

A Dice-együttható szintén a $[0, 1]$ intervallumon mozog. Orvosi képfeldolgozásban különösen népszerű, mivel közvetlenebbül fejezi ki a pozitív találatok (polip pixelek) arányát, és kevésbé érzékeny a háttér méretére. Fontos megjegyezni, hogy ugyanazon szegmentációs eredmény esetén a Dice érték mindig nagyobb vagy egyenlő, mint az IoU érték ($Dice \geq IoU$), mivel a nevezője kisebb ($|P| + |G|$ vs. $|P \cup G|$).

2.1.3 Valós idejű rendszerek teljesítménymutatói

Egy videófeldolgozó rendszer hatékonyságának mérésére két fő mérőszámot, a throughputot (áteresztőképesség) és a látenciát (késleltetés) alkalmazzuk. Míg a throughput azt mutatja meg, hogy egységnyi idő alatt hány képkockát képes a rendszer feldolgozni (FPS), addig a látencia azt az időintervallumot jelöli, amely a képkocka beérkezése és a feldolgozott eredmény képernyőn való megjelenése között telik el. Orvosi környezetben, különösen az intervenciók eljárásoknál, a látencia kritikusabb tényező lehet, mint a maximális FPS. Ha a rendszer késleltetése meghaladja a 100-200 milliszekundumot, az zavarhatja az orvos kéz-szem koordinációját, mivel az eszköz mozgatása és a képernyőn látott reakció között érezhető csúszás lép fel ("rubber band effect"). Ezért a fejlesztés során a cél nem pusztán a magas FPS elérése, hanem a stabil, alacsony késleltetésű (low-latency) működés biztosítása volt.

2.2 Irodalmi áttekintés

2.2.1 Mélytanulás az orvosi képfeldolgozásban

Az orvosi képfeldolgozás területén a mélytanulás (Deep Learning) megjelenése forradalmi változásokat hozott, különösen a Fully Convolutional Networks (FCN) és az U-Net architektúra bevezetésével. Az U-Net, szimmetrikus enkóder-dekóder felépítésével és skip connection megoldásaival, máig az orvosi szegmentáció de facto standardjának számít, bár a valós idejű alkalmazásokban korlátokba ütközhet a magas paraméterszám és a számítási komplexitás miatt. A hagyományos gépi tanulási módszerekkel (pl. SVM, Random Forest) szemben, ahol a jellemzőket (szín, textúra, alak) manuálisan kellett definiálni ("hand-crafted features"), a mélytanuló modellek (CNN) képesek az adatokból automatikusan kinyerni a releváns, hierarchikus reprezentációkat. Ez különösen előnyös a polipok esetében, amelyek megjelenése rendkívül változatos lehet, és nehéz őket egyszerű szabályokkal leírni.

2.2.2 A DeepLab model család és az ASPP modul

A Google által fejlesztett DeepLab model család jelentős előrelépést hozott a szegmentációs feladatokban, elsősorban a "dilated" (vagy atrous) konvolúciók bevezetésével, amelyek lehetővé teszik a receptív mező növelését a felbontás drasztikus csökkentése nélkül. A DeepLabV3+ egyik kulcseleme az Atrous Spatial Pyramid Pooling (ASPP) modul, amely képes a képek különböző léptékű jellemzőit párhuzamosan kinyerni, így hatékonyan kezeli a polipok szélsőséges méretbeli változatosságát. Az ASPP modul több párhuzamos atrous konvolúciós réteget tartalmaz eltérő rátákkal (dilation rates), amelyek kimeneteit összefűzve a hálózat egyszerre látja a finom részleteket és a globális kontextust. A V3+ verzió ezt egy egyszerű, de hatékony dekódoló ággal egészíti ki, amely a felskálázott jellemzőket kombinálja az alacsony szintű élinformációkkal, így pontosabb határokat eredményez, mint elődei.

2.2.3 Valós idejű videófeldolgozás Python környezetben

Bár a Python a Data Science elsődleges nyelve, a valós idejű, nagy teljesítményű videófeldolgozás során komoly akadályt jelenthet a Global Interpreter Lock (GIL), amely korlátozza a szálak párhuzamos futtatását. A szakirodalom különböző megoldásokat javasol ezen korlátok megkerülésére, a multiprocessing modul használatától kezdve a számításigényes feladatok GPU-ra (CUDA) történő kiszervezéséig. A gyakorlatban a képfeldolgozás (OpenCV) és a neurális hálózat futtatása (PyTorch) C++ alapokon nyugszik, így a Python "csak" a vezérlést végzi ("glue code"). A GIL hatása leginkább a Python-szintű hurkoknál és adatmozgatásoknál jelentkezik, ezért kritikus a vektorizált műveletek (NumPy) használata és az aszinkron I/O műveletek megfelelő szervezése a várakozási idők minimalizálása érdekében.

2.3 Alkalmazott eszközök és technológiák

2.3.1 Programozási környezet

A fejlesztés során a Python 3.x verziója került kiválasztásra, amely széleskörű könyvtár-támogatottsága miatt ideális környezet a prototípus-fejlesztéshez és a kutatáshoz. A mélytanuló modellek implementálásához a PyTorch keretrendszer szolgált alapul, amely dinamikus számítási grájával és intuitív API-jával rugalmas kísérletezést tesz lehetővé. A PyTorch "eager execution" modellje nagyban megkönnyíti a hibakeresést és a modell viselkedésének lépésről-lépésre történő követését, szemben a statikus gráfokat használó rendszerekkel (pl. TensorFlow 1.x). Emellett a kiterjedt közösségi támogatás és a kész, előtanított modellek (torchvision) elérhetősége is mellette szólt.

2.3.2 Képfeldolgozó könyvtárak

A videófolyamok kezeléséhez és az alapvető képmanipulációs műveletekhez az ipari szabványnak számító OpenCV könyvtár, míg a hatékony mátrixműveletekhez a NumPy került alkalmazásra. Az adatok előkészítése és a modell robusztusságának növelése érdekében az Albumentations könyvtár szolgált, amely gyors és változatos augmentációs technikákat kínál. Az Albumentations kifejezetten a gépi tanulási feladatokra optimalizált, és jelentősen gyorsabb, mint a hagyományos megoldások (pl. PIL), miközben gazdag transzformációs készletet (rugalmas torzítás, színcsatorna manipulációk) biztosít a tanítóadatok dúsításához.

2.3.3 Hardveres környezet

A projekt során két eltérő hardverkonfiguráció került alkalmazásra: egy nagy teljesítményű munkaállomás a modell tanításához, és egy hordozható számítógép a valós idejű futtatás demonstrálásához.

Tanító környezet

A mélytanuló modell számításigényes tanítási fázisa egy dedikált munkaállomáson zajlott, amely biztosította a szükséges párhuzamosítási kapacitást a gyors konvergenciához.

- **GPU:** NVIDIA GeForce RTX széria (CUDA támogatással).
- **Feladat:** Adat-előfeldolgozás, augmentáció, a DeepLabV3+ hálózat tanítása és validációja.

A valós idejű videójel fogadásához ebben a környezetben egy speciális, alacsony késleltetésű HDMI capture kártya is beépítésre került, amely biztosította a nyers videóadatok folyamatos áramlását a számítógép felé a kezdeti tesztek során.

Tesztelési és futtatási környezet

A kész szoftver teljesítménytesztelése és a hordozhatóság demonstrálása egy ASUS Zenbook 14 típusú ultrabookon történt. A cél annak igazolása volt, hogy az optimalizált modell (MobileNetV2 backbone) gyengébb hardveren is képes a közel valós idejű működésre.

- **Processzor:** AMD Ryzen 7 5825U (8 mag, 16 szál, alapórajel 2.00 GHz, Boost akár 4.5 GHz).
- **Memória:** 16 GB LPDDR4X (4266 MT/s sebesség).
- **Grafikus vezérlő:** Integrált AMD Radeon Graphics (megosztott rendszermemóriával).
- **Tárhely:** 477 GB SSD.

Az 5. fejezetben bemutatott teljesítménymérések (pl. a 350 ms-os válaszidő) ezen a konfiguráción születtek, igazolva a rendszer hatékonyságát dedikált videokártya nélküli környezetben is.

fejezet 3

A szegmentáló modell tervezése és implementációja

Becsült terjedelem: 6-8 oldal

3.1 Adatbázisok és előkészítés

3.1.1 Adatbázisok bemutatása

A modell tanításához és validálásához két fő forrásból származó adatbázis került felhasználásra. Az első a nyilvánosan elérhető, standardizált Kvasir-SEG dataset [3], amely 1000 darab annotált polip képet tartalmaz változatos patológiai esetekkel és hozzájuk tartozó bináris szegmentációs maszkokkal; ez az adatkészlet a polip szegmentálási kutatások egyik leggyakrabban használt benchmarkja. A második forrás a Széchenyi István Egyetem (SZE) kutatócsoportja által gyűjtött SZE Dataset, amely specifikus klinikai körülmények között készült endoszkópos felvételeket tartalmaz. Az SZE Dataset Excel-alapú annotációval rendelkezik, amely minden képhez egyedi páciens-azonosítót (ID), osztálycímét (CLASS) és minőségi jelölést (QUALITY) rendel. A két adatbázis egyesítése során a Kvasir-SEG képek egyedi azonosítót kaptak (`kvasir_` prefix), biztosítva az ütközésmentes páciensazonosítást a későbbi adatfelosztás során.

3.1.2 Adattisztítás és annotációs kihívások

Az adatok feldolgozása során az egyik legnagyobb kihívást a hibás vagy pontatlan annotációk, valamint a zajos, diagnosztikai szempontból értékelhetetlen képkockák szűrése jelentette. Az SZE Dataset fájlnev-konvenciójában a 6. pozíción álló betű (E vagy R) jelöli a kép minőségét, ami lehetővé tette az automatikus előszűrést. A manuális átvizsgálás során eltávolításra kerültek azok a maszkok, amelyek nem fedték pontosan a léziót, illetve korrigálásra

kerültek a széket vagy buborékokat tévesen polipként jelölő annotációk (fals pozitívok). Az endoszkópos képalkotás további speciális kihívásai – például a nyálkahártya nedves felületéről visszaverődő fény (speculáris tükröződések) és a széles látószögű lencse által okozott hordótorzítás – szintén befolyásolhatják a szegmentálás pontosságát, ezért a tanítóhalmaz összeállításakor ezekre az artefaktumokra is kiemelt figyelmet kellett fordítani.

3.1.3 Augmentációs stratégiák

Mivel az orvosi képalkotó adatbázisok mérete viszonylag korlátozott, a modell általánosító képességének növelése és a túltanulás elkerülése érdekében adat-augmentációs stratégia került alkalmazásra. A tréning adatokra a következő transzformációs lánc (pipeline) vonatkozott: véletlenszerű léptékváltoztatás (`ExtRandomScale`, $0.5\text{--}2.0\times$ tartomány), véletlenszerű horizontális tükrözés (`ExtRandomHorizontalFlip`), véletlenszerű kivágás 513×513 pixel méretre (`ExtRandomCrop`), véletlenszerű forgatás (`ExtRandomRotation`), majd a tenzorralakítás (`ExtToTensor`) és az ImageNet normalizáció (`ExtNormalize`, $\text{mean}=[0.485, 0.456, 0.406]$, $\text{std}=[0.229, 0.224, 0.225]$). A validációs és tesztelési fázisban ezzel szemben kizárólag determinisztikus transzformációk kerültek alkalmazásra: átméretezés 513 pixelre (`ExtResize`), középre vágás (`ExtCenterCrop`), tenzorralakítás és normalizáció – augmentáció nélkül. A léptékváltoztatás és a forgatás biztosítja, hogy a hálózat megtanulja felismerni a polipokat különböző méretben és orientációban, míg a tükrözés a szimmetria-invarianciát erősíti.

3.2 Modell architektúra

3.2.1 DeepLabV3+ kiválasztásának indoklása

A szegmentációs architektúra kiválasztásakor több modell összehasonlítása történt. Az orvosi szegmentálásban korábban domináns U-Net [4] szimmetrikus kódoló-dekódoló struktúrájával és skip connection megoldásaival statikus orvosi képeken kiválóan teljesít, azonban a kolonoszkópia dinamikus környezetében és a polipok szélsőséges méretbeli változatossága (néhány millimétertől több centiméterig) esetén a hagyományos konvolúciós rétegek fix receptív mezője korlátot jelent. A Google által fejlesztett DeepLab modellcsalád [5] éppen ezt a többléptékű kontextus-megragadási problémát célozta meg: a DeepLabV1/V2 az atrous (dilatált) konvolúciót és az Atrous Spatial Pyramid Pooling (ASPP) modult vezette be, a DeepLabV3 a globális átlagolást (global average pooling) illesztette az ASPP-hez, míg a DeepLabV3+ egy dekódoló modullal egészítette ki a rendszert, amely a felskálázott szemantikai információt ötvözi a gerinchálózat korábbi rétegeinek nagy felbontású, alacsony szintű jellemzőivel. Ez a kombináció képes visszaállítani a polipok éles határvonalait, ami kritikus a pontos méretmeghatározáshoz. A szakirodalmi benchmark adatok is alátámasztják a vá-

lasztást: a CVC-ClinicDB adatkészleten az U-Net átlagosan $\sim 89\text{--}90\%$ Dice-együtthatót ért el, míg a DeepLabV3+ MobileNetV2 gerinchálózattal $93\text{--}96\%$ Dice értéket produkált [6].

3.2.2 MobileNetV2 backbone előnyei

Az alacsony késleltetésű működés követelménye megkövetelte a DeepLabV3+ modell gerinchálózatainak (backbone) gondos megválasztását. A robusztus, de számításigényes ResNet vagy Xception helyett a MobileNetV2 [7] került alkalmazásra, amely a mélységi szeparálható konvolúció (depthwise separable convolution) technikáján alapul. Ez a megoldás a standard konvolúciót két lépésre bontja: egy térbeli szűrésre (depthwise) és egy csatorna-kombinációra (pointwise), ami tipikus 3×3 -as kernel esetén 8–9-szeres számítási megtakarítást eredményez. A MobileNetV2 további kulcsinnovációja a fordított reziduális blokk (inverted residual block), amely “Narrow-Wide-Narrow” struktúrát alkalmaz: először 1×1 konvolúcióval kiterjeszti a csatornák számát (általában 6-szorosára), majd a kiterjesztett téren végzi a 3×3 -as mélységi konvolúciót, végül visszacsökkenti a dimenziót. A kimeneti bottleneck rétegen lineáris aktiváció kerül alkalmazásra a nem-lineáris aktiváció (ReLU) helyett, ami megőrzi az alacsony dimenziós jellemzők finom információtartalmát. A gyakorlatban ez a kombináció drasztikus különbséget jelent: a MobileNetV2 mindössze 2,1–5,8 millió paraméterrel rendelkezik a ResNet50 $\sim 40\text{--}50$ millió vagy az Xception $\sim 54,7$ millió paraméterével szemben, és GPU-n 40–50 FPS sebességet ér el, míg a ResNet50 csupán $\sim 20\text{--}25$ FPS-t. A jelen implementációban az output stride értéke 16-ra (output_stride=16) lett beállítva, ami azt jelenti, hogy a MobileNetV2 utolsó blokkjaiban a stride=2 helyett stride=1 kerül alkalmazásra, és a receptív mező fenntartása atrous konvolúcióval valósul meg.

3.2.3 Veszteségfüggvény és optimalizáció

A célfüggvény (loss function) megválasztása kulcsfontosságú a konvergencia szempontjából. A feladathoz a CrossEntropyLoss került alkalmazásra, amely a két osztályos (háttér és polip, num_classes=2) szegmentáláshoz jól illeszkedik: az egyes pixelek valószínűségi eloszlását hasonlítja össze a valódi címkékkel, és hatékonyan bünteti a tévesztéseket mindkét irányban. Az optimalizáláshoz az Adam algoritmus szolgált (lr=0.001), amely adaptív tanulási rátájának köszönhetően gyorsabb konvergenciát biztosít a hagyományos SGD-hez képest. A tanulási ráta finomhangolására a ReduceLROnPlateau ütemező (scheduler) lett beállítva: amennyiben a validációs IoU érték 3 egymást követő epoch során nem javul (patience=3), az ütemező a tanulási rátát tizedére csökkenti (factor=0.1), ezzel segítve a modell finomabb konvergenciáját a tanítás későbbi fázisaiban.

3.3 Tanítási és validációs stratégia

3.3.1 Transzfer tanulás

A rendelkezésre álló korlátozott mennyiségű orvosi adat miatt a “nulláról” történő tanítás helyett a transzfer tanulás (transfer learning) módszere került alkalmazásra. A MobileNetV2 gerinchálózat az ImageNet adatbázison előtanított, majd a PASCAL VOC szegmentációs feladatra továbbtanított súlyokkal lett inicializálva (amit a checkpoint fájlnev `_voc_os16` utótagja is jelez). Az előtanított súlyok biztosítják, hogy a hálózat alacsony szintű rétegei már a tanítás kezdetén is képesek legyenek az alapvető vizuális jellemzők (élek, textúrák, színátmenetek) kinyerésére, ami drasztikusan felgyorsítja a konvergenciát. A finomhangolás (fine-tuning) során a teljes hálózat súlyai módosultak – a gerinchálózat rétegei nem kerültek befagyasztásra –, lehetővé téve, hogy a modell adaptálódjon az endoszkópos polipképek specifikus jellemzőihez.

3.3.2 A StratifiedGroupKFold jelentősége

A modell validációja során kiemelt figyelmet kellett fordítani az adatszivárgás (data leakage) elkerülésére, amely akkor léphet fel, ha ugyanazon betegről készült, egymáshoz nagyon hasonló képkockák kerülnek a tanító és a validációs halmazba is. Ennek megelőzésére a standard véletlenszerű szétválasztás helyett a StratifiedGroupKFold módszer került alkalmazásra (`n_splits=100`, `shuffle=True`, `random_state=42`). A “Group” komponens biztosítja, hogy egy páciens összes képe ugyanabba a halmazba kerüljön (csoportosítás az ID oszlop alapján), míg a “Stratified” komponens a CLASS oszlop alapján gondoskodik arról, hogy a polip jelenlétének aránya kiegyensúlyozott maradjon a halmazok között. Az adatfelosztás aránya: 70% tréning, 15% validáció, 15% teszt. Az SZE Dataset felosztásának konkrét eredménye: a tréning halmaz 476 képet (272 páciens), a validációs halmaz 102 képet (83 páciens), a teszthalmaz pedig 495 képet (84 páciens) tartalmaz. A Kvasir-SEG adatkészlet teljes egészében a tréning halmazhoz került hozzáadásra, mivel az nem tartalmaz páciens-szintű azonosítókat, így az adatszivárgás kockázata nem áll fenn.

3.3.3 Hiperparaméterek

Az optimális hiperparaméterek meghatározása iteratív kísérletezés útján történt, a pontosság és a tanítási idő közötti legjobb kompromisszum megtalálása céljából. A bemeneti képek felbontása 513×513 pixelben került rögzítésre, összhangban a DeepLabV3+ architektúra alapértelmezett bemeneti méretével. A kötegméret (batch size) 4-re lett beállítva (`batch_size=4`), figyelembe véve a rendelkezésre álló GPU memória kapacitását. A tanítás összesen 15 epoch-on (`num_epochs=15`) keresztül futott, explicit korai megállási (early stopping) mechanizmus nélkül. Ehelyett a legjobb modell mentése a validációs IoU érték

alapján történt: amennyiben egy epoch végén a validációs IoU meghaladta az addigi legjobb értéket, a modell súlyai automatikusan elmentődtek. A végső, legjobb teljesítményt nyújtó modell a 12. epoch-ban érte el a maximális validációs IoU-t, amelynek elnevezése: `polyp_trained_e12_best_deeplabv3plus_mobilenet_voc_os16.pth`. A kimeneti osztályok száma 2 (`num_classes=2`): háttér és polip, az output stride értéke pedig 16 (`output_stride=16`).

fejezet 4

A szoftveralkalmazás tervezése és fejlesztése

Becsült terjedelelem: 6-8 oldal

4.1 Rendszerarchitektúra és követelmények

4.1.1 Funkcionális követelmények

A fejlesztett szoftverrel szemben támasztott legfontosabb funkcionális követelmény, hogy képes legyen folyamatosan fogadni a HDMI forrásból érkező videójelet (vagy webkamera képet, videófájlt), és biztosítsa a valós idejű szegmentálás képességét felhasználói beavatkozásra. A rendszernek támogatnia kell a “Capture & Analyze” (Rögzítés és Elemzés) funkciót, amely egy dedikált gomb vagy billentyű (Space) lenyomására a pillanatnyi képkockát kiemeli a videófolyamból, és lefuttatja rajta a szegmentáló és osztályozó algoritmusokat. Az eredményt vizuálisan vissza kell csatolni a felhasználónak: meg kell jeleníteni a hálózat által látott (előfeldolgozott, középre vágott) képet és a szegmentált maszkot az eredeti képre vetítve. A szegmentált polip régió (ROI) továbbá átadható a klasszifikátornak – jelenleg demonstrációs célú `dummy_classify` metódus –, amely szövettani becslést végző algoritmusoknak, további elemzésre vagy audit célokra. Emellett a rendszernek naplóznia kell a futási teljesítményadatokat (előfeldolgozás, inferencia, utófeldolgozás ideje) egy CSV fájlba (`performance_log.csv`), amely lehetővé teszi a későbbi elemzést és a rendszer teljesítményének nyomon követését.

4.1.2 Nem-funkcionális követelmények

Mivel a rendszer potenciálisan orvosi beavatkozások során kerülne alkalmazásra, a nem-funkcionális követelmények közül a stabilitás és a megbízhatóság élvez prioritást: a szoftver

nem fagyhat le és nem omolhat össze működés közben, különösen a nagy számításigényű inferencia alatt. Ennek érdekében a felhasználói felületet (GUI) kezelő szálnak el kell különülnie a videófeldolgozó és az elemző szálaiktól – a VideoCaptureThread daemon szálon fut, ahol folyamatosan olvassa a forrást, míg a főszál kezeli a GUI-t és az igény szerinti elemzést. További követelmény a modularitás: a MedicalAnalyzer osztály cseréjével vagy frissítésével a keretrendszer újraindítása nélkül lehessen újabb, pontosabb modelleket integrálni. A szoftvernek hardver-függetlennek kell lennie, azaz képesnek kell lennie futni CPU-n és GPU-n is, automatikusan detektálva a rendelkezésre álló erőforrásokat (`torch.device('cuda' if torch.cuda.is_available() else 'cpu')`).

4.1.3 Heterogén keretrendszerek integrációja

A kutatás során felmerült az a kiemelt architektúráis igény, hogy a rendszer ne csupán egyetlen mélytanulási keretrendszerhez (pl. PyTorch) legyen kötve, hanem képes legyen heterogén környezetben működni. Bár a szegmentációs feladatot PyTorch alapú DeepLabV3+ és U-Net modellek végzik, a szoftvert úgy kellett kialakítani, hogy zökkenőmentesen fogadhasson TensorFlow (Keras) alapú klasszifikátor modelleket is. Ez a megközelítés maximalizálja a rendszer rugalmasságát, hiszen lehetővé teszi, hogy külső kutatócsoportok által – akár eltérő keretrendszerben – fejlesztett szövettani osztályozó hálózatokat (pl. egy TensorFlow-ban tanított ResNet50V2) utólagosan, a kód minimális módosításával csatlakoztassunk. A két eltérő keretrendszer egyidejű memóriában tartása komoly kihívást jelent a korlátozott GPU VRAM (Video RAM) miatt. Ennek kiküszöbölésére a TensorFlow futtatását a tervezés során szándékosan a központi processzorra (CPU) korlátoztuk (`tf.config.set_visible_devices([], 'GPU')`). Mivel a klasszifikáció csupán egy kisméretű (jellemzően 512×512 pixeles), egy-szeri képkivágáson fut le gombnyomásra, a CPU-n történő végrehajtás nem okoz észrevehető teljesítménycsökkenést, cserébe garantálja, hogy a GPU memóriája kizárólag a nagy erőforrás-igényű PyTorch szegmentáció számára maradjon fenntartva.

4.1.4 Adatfolyam diagram

A rendszer működésének átláthatósága érdekében elkészült az adatfolyam (Data Flow) diagram, amely vizuálisan ábrázolja az információ útját a fizikai bemenettől a megjelenítőig. Az ábra részletesen bemutatja, hogyan kerül a nyers frame a capture kártyáról (vagy videófájlból, szimulációs képmappából) a VideoCaptureThread pufferébe, onnan a GUI frissítési ciklusába élő képként. Elemzés kérésekor a frame a MedicalAnalyzer modulba kerül, ahol átesik az előfeldolgozáson (BGR→RGB konverzió, PIL Image, átméretezés 513×513 -ra, középre vágás, ImageNet normalizáció), majd a GPU-ra kerül inferenciára. A kapott valószínűségi térképet (probability map) a rendszer argmax után bináris maszkká alakítja, de-normalizálja a bemeneti tenzort a vizualizációhoz, majd a polip maszk alapján az eredeti kép tartalmát megjeleníti (háttér fekete). A végső megjelenítés előtt a klasszifikátor (jelen-

leg dummy_classify) predikciót ad, amelyet a GUI szövegesen is megjelenít a három panel mellett.

4.2 Implementációs részletek

4.2.1 Videó rögzítő modul

A videófolyam zökkenőmentes kezelésére egy dedikált, aszinkron működésű rögzítő osztály (VideoCaptureThread) került implementálásra az OpenCV VideoCapture moduljára alapozva. A frame-eldobások és a GUI szaggatásának elkerülése érdekében a rögzítés külön szálon (daemon thread) fut, amely folyamatosan olvassa a hardveres puffert, és mindig csak a legutolsó érvényes képkockát tárolja el egy threading.Lock által védett változóban. A read() metódus lock-kal védett másolatot ad vissza, így a fő alkalmazás (App) mindig a legfrissebb elérhető adatot kapja meg várakozás nélkül (non-blocking read), miközben a capture szál háttérben tovább olvas. Videófájl módban a set_position(percent) lehetővé teszi a pozíció ugrást (slider), lock-kal védve a seek műveletet. A modul támogatja a szimulációs módot is: fizikai kamera hiányában az images/ mappából tölt be véletlenszerű tesztképeket, szűrve a releváns páciens azonosítókra (allowed_ids, pl. PI009, PI015, ...), 2 másodpercenként váltva – lehetővé téve a fejlesztést klinikai környezet nélkül is.

4.2.2 Inferáló motor

A mélytanuló modell futtatásáért felelős inferáló motor (MedicalAnalyzer) optimalizálása során a PyTorch eval() módja és a torch.no_grad() kontextus került alkalmazásra a felesleges gradiens-számítások és a számítási gráf építésének kikapcsolására, ami jelentősen csökkenti a memóriaigényt és gyorsítja a futást. A bemeneti képek előkészítése (preprocessing) pontosan követi a tanítás során használt lépéseket: a utils.ext_transforms modul ExtResize(513), ExtCenterCrop(513), ExtToTensor és ExtNormalize (ImageNet mean/std) transzformációit alkalmazza, biztosítva a konzisztens eredményeket. A kimeneti maszkok vizualizációjához a rendszer inverz transzformációt (denormalizáció) végez: a bemeneti tenzort (C, H, W) visszaalakítja NumPy tömbökké, transzponálja (H, W, C)-re, majd visszaállítja az eredeti színeket a std * tensor + mean képlettel és np.clip-pel. A polip maszk alapján az eredeti kép tartalma megjelenik, a háttér fekete marad – alpha blending helyett egyszerű maszkolás, mivel a polip textúrája és felülete látható maradjon.

4.2.3 Konfidencia-alapú döntéstámogatás és a klasszifikátor interfész

A rendszer egyik kulcsfontosságú eleme a modularitás jegyében kialakított klasszifikációs logika és a konfidencia-alapú döntéstámogatás. Amikor a szegmentáló modell polipot detektál (a maszk nem üres), a rendszer egy _extract_roi metódus segítségével meghatározza a

polipot befoglaló legkisebb téglalapot (bounding box), majd ezt a képrészletet (Region of Interest) kivágja, átméretezi és átadja a másodlagos, TensorFlow alapú osztályozó hálózatnak. Jelen implementációban egy bináris klasszifikációra betanított ResNet50V2 hálózat végzi a szövettani besorolást. A klasszifikátor kimenete egyetlen sigmoid neuron, amely a malignitás (rosszindulatúság) valószínűségét adja meg: 0, 5 feletti érték esetén a képletet *Malignant (JNET 2a/2b/3)* kategóriába, alatta pedig *Benign (JNET 1)* kategóriába sorolja a rendszer.

A döntéstámogatás megbízhatóságának növelése érdekében a szoftver két különböző konfidenciaértéket is kiszámít és megjelenít az orvos számára:

- **Szegmentációs konfidencia:** A hálózat (DeepLabV3+ esetén softmax, U-Net esetén sigmoid) nyers kimeneti valószínűségi térképéből (probability map) számítódik. A rendszer veszi a bináris maszkon belül eső (polipként detektált) pixelek valószínűségi értékeinek átlagát. Ez megmutatja, hogy a modell átlagosan mennyire "biztos" abban, hogy a kijelölt terület valóban polip (pl. 78% vagy 95%).
- **Klasszifikációs konfidencia:** A TensorFlow modell sigmoid kimenetéből származó valószínűség, amely kifejezi az adott szövettani osztályba (Benign vagy Malignant) tartozás bizonyosságát.

Ezek a metrikák kritikus fontosságúak a klinikai gyakorlatban: egy alacsony szegmentációs konfidencia (pl. 55%) vizuális fals pozitív (pl. fénybecsillanás vagy székletmaradvány) gyanúját keltheti, ami felülbírálatra ösztönzi a vizsgáló orvost.

4.3 Felhasználói felület terve

4.3.1 Orvosi felhasználói élmény

A felhasználói felület tervezésekor az orvosi környezet sajátosságai kerültek előtérbe: a szoftvernek a háttérben kell maradnia, és csak akkor szabad vizuális inger adnia, ha az valóban szükséges – ne vonja el a figyelmet a műtétről vagy a vizsgálat biztonságos lebonyolításáról. A zavaró tényezők minimalizálása érdekében sötét téma (dark mode, #2c3e50, #34495e) került alkalmazásra, ami csökkenti a szem fáradását a gyakran sötétített endoszkópos vizsgállókban. A kezelőfelület minimalista, a gombok nagy méretűek és jól elkülönülnek, támogatva az érintőképernyős használatot is. A "Capture & Analyze" funkció a Space billentyűre is map-elve van, ami lehetővé teszi, hogy az orvos egy lábpedál segítségével, kézhasználat nélkül indítsa el az elemzést munka közben.

4.3.2 Vizualizációs megoldások

A képernyő elrendezése három fő panelre oszlik, segítve az orvost a döntéshozatalban. A bal oldali panelen ("Live Feed") látható az élő, módosíthatatlan videókép, 400 px magasságúra

skalázva. A középső panel ("Network Input (Center Crop)") mutatja a neurális hálózat bemenetét – a középre vágott, 513×513 -ra méretezett képet –, ami segít ellenőrizni, hogy a polip a látótérben van-e. A jobb oldali panel ("Polyp Segmentation") jeleníti meg a szegmentált eredményt: a polip maszk alapján az eredeti kép tartalma megjelenik, a háttér fekete marad, így a polip textúrája és felülete látható marad. A rendszer közvetlenül a szegmentált kép alatt nagy méretű betűkkel szövegesen is megjeleníti a szegmentációs konfidenciát, a klasszifikációs eredményt és annak valószínűségét (pl. "Szegm.: 88.5% | Benign (JNET 1) (92.1%)"), azonnali és jól olvasható döntéstámogatást nyújtva ezzel.

4.3.3 Adatgyűjtés és automatikus mentés

Egy orvosi döntéstámogató rendszer esetében kiemelt jelentőséggel bír a vizsgálatok dokumentálhatósága, mind a későbbi orvosi audit, mind az utólagos modell-továbbfejlesztés (adatgyűjtés) céljából. Ennek kiszolgálására az alkalmazás egy robusztus mentési funkcióval bővült, amely a GUI-n keresztül szabályozható. A felhasználó két mód közül választhat:

- **Manuális mentés:** Ez az alapértelmezett viselkedés. A felhasználó a "Capture & Analyze" lenyomásakor csak a képernyőn látja az eredményt. Ha a talált elváltozás diagnosztikai vagy kutatási szempontból értékes, a "Mentés" gombra kattintva rögzítheti azt.
- **Auto mentés:** Ebben a módban minden egyes alkalommal, amikor az orvos (a gép vagy a lábpedál segítségével) elemzést indít, a rendszer a háttérben automatikusan és aszinkron módon elmenti az eredményeket, anélkül, hogy a vizsgálat menetét megakasztaná.

A mentési folyamat (`_save_results`) másodperc-pontosságú időbélyegekkel (YYYYMMDD_HHMMSS) dolgozik. Minden mentési esemény során egy közös mappába (`saved_results`) generálódik három képfájl: az eredeti videóképkocka, a hálózatnak beadott középre vágott bemenet, valamint a polip-maszkolt eredmény. Technikai szempontból fontos részlet, hogy a Windows operációs rendszeren az OpenCV `cv2.imwrite` függvénye nem támogatja a speciális karaktereket (pl. magyar ékezetek) tartalmazó fájlnévonalakat. Ezt a problémát a memóriapufferen keresztüli kódolással (`cv2.imencode`) és natív fájlba írással hidaltuk át. A képek mellé a rendszer egy strukturált JSON formátumú metaadat fájlt (`metadata.json`) is elhelyez, amely eltárolja az időbélyeget, a klasszifikátor kimeneti osztályát, a szegmentációs konfidenciát és az osztályozási valószínűséget. Ez az adatstruktúra ideális alapot biztosít a későbbi statisztikai elemzésekhez és a gépi tanulási modellek iteratív újratanításához.

fejezet 5

Eredmények és kiértékelés

Becsült terjedelem: 4-5 oldal

5.1 A szegmentáló modell teljesítménye

5.1.1 Kvantitatív eredmények

A kifejlesztett DeepLabV3+ alapú szegmentáló modell pontossága az SZE teszhalmazon végzett független mérésekkel került igazolásra. A modell kiértékelésére a legjobb validációs teljesítményt nyújtó checkpoint (epoch 38) került kiválasztásra, amely a validációs halmazon 84,49% átlagos IoU-t ért el. A teszhalmazon végzett mérések megerősítették a modell robusztusságát: a rendszer 83,31% átlagos IoU és 90,40% átlagos Dice score értéket produkált (lásd 5.1. táblázat).

A 90,40%-os Dice score érték a szakirodalomban is kiváló szegmentációs teljesítménynek számít, különösen figyelembe véve, hogy a MobileNetV2 egy erőforrás-takarékos, valós idejű futtatásra optimalizált gerinchálózat. Az eredmények azt mutatják, hogy a modell nemcsak a polipok jelenlétét detektálja megbízhatóan, hanem azok alakját és kiterjedését is nagy pontossággal határolja körül. A teszt és a validációs eredmények közötti minimális eltérés (~1,18% IoU csökkenés) arra utal, hogy a modell jól általánosít, és nem történt jelentős túlfitting (overfitting) a tanító adatokon. Ez a stabilitás kritikus fontosságú a klinikai alkalmazhatóság szempontjából, ahol a rendszernek ismeretlen, változatos endoszkópos felvételeken is megbízhatóan kell teljesítenie.

5.1. táblázat: A kiválasztott szegmentáló modell kvantitatív eredményei (SZE validáció és teszt halmaz).

Epoch	Val IoU [%]	Test IoU [%]	Test Dice [%]
38	84,49	83,31	90,40

5.1.2 Kvalitatív elemzés

A számszerű eredmények mellett a vizuális kiértékelés is megerősíti a modell hatékonyságát. A sikeres szegmentálásoknál megfigyelhető, hogy a hálózat pontosan követi a polipok szabálytalan kontúrjait, és még a nyálkahártyával színben alig eltérő, lapos (sessile) elváltozásokat is képes elkülöníteni a környezettől. A DeepLabV3+ architektúra ASPP moduljának köszönhetően a modell a különböző méretű polipokat – az apró, néhány milliméteres képletektől a nagyobb adenómákig – egyaránt felismeri.

Ugyanakkor a tesztelés során azonosításra kerültek tipikus hibaforrások is. A leggyakoribb tévesztést a speculáris tükröződések (fénybecsillanások) és a székletmaradványok vagy buborékok okozzák, amelyeket a modell esetenként tévesen polipként (fals pozitív) jelöl meg. Szintén kihívást jelentenek a kép szélein elhelyezkedő, részlegesen látható polipok, illetve a rossz megvilágítású vagy életlen (motion blur) képkockák. Ezek a hibák azonban jellemzően lokálisak és rövid ideig tartanak, így a videófolyam folyamatos elemzése során a rendszer összességében stabil detektálást nyújt.

5.2 Rendszerteljesítmény mérése

5.2.1 Megjelenítési sebesség és igény szerinti elemzési idő

A rendszer teljesítményét két szempont szerint értékeltem: a folyamatos videófolyam megjelenítési sebessége (FPS) és a felhasználói beavatkozásra (gombnyomás) történő elemzés válaszideje (latency).

Az élő videókép megjelenítését a szoftver dedikált szálon (thread) végzi, amely leválasztja a GUI frissítését a capture folyamatról. A mérések során a rendszer stabilan tartotta a 30 FPS megjelenítési sebességet, biztosítva a folyamatos, akadásmentes vizuális visszacsatolást az orvos számára.

A gombnyomásra történő elemzés ("Capture & Analyze") teljes körű válaszidejét 120 mérési pont átlagolásával határoztam meg. A mérések magukban foglalták az előfeldolgozást (képkivágás, tenzorral alakítás), a neurális háló inferenciáját és az utófeldolgozást (maszk színezése, megjelenítése). Az átlagos teljes válaszidő **351,36 ms** (szórás: $\pm 51,01$ ms) volt. Ennek komponensei az alábbiak szerint oszlanak meg:

- **Előfeldolgozás:** átlagosan 20,00 ms.
- **Inferencia (DeepLabV3+):** átlagosan 307,71 ms.
- **Utófeldolgozás és megjelenítés:** átlagosan 23,65 ms.

Bár az eredmény meghaladja a szigorúan vett valós idejű (real-time) videófeldolgozás 33-

40 ms-os keretidejét, a rendszer tervezett felhasználási módjában – ahol az elemzés igény szerint, statikus pillanatfelvételen történik – ez az érték elfogadható. A felhasználó számára a gombnyomás és az eredmény megjelenése közötti 0,35 másodperces késleltetés szinte azonnalnak érződik, és nem akadályozza a klinikai döntéshozatalt. A futásidő legnagyobb részét (87%) az inferencia teszi ki, ami erősebb dedikált GPU alkalmazásával vagy a modell további optimalizálásával (pl. TensorRT, kvantálás) jelentősen csökkenthető lenne.

5.2.2 Teljes késleltetés és erőforrás-használat

A rendszer stabilitását igazolja, hogy a memóriahasználat a hosszú távú futtatás során is konzisztens maradt, memória-szivárgás nem lépett fel. A GPU memória (VRAM) kihasználtsága az inferencia alatt érte el a csúcst, de a MobileNetV2 alacsony paraméterszáma miatt ez nem jelentett szűk keresztmetszetet még a korlátozottabb erőforrásokkal rendelkező tesztkörnyezetben sem.

5.3 Diszkusszió

5.3.1 A gombnyomásra történő elemzés korlátai

Az eredmények elemzése során a gombnyomásra történő elemzés működési módjának sajátos korlátai váltak nyilvánvalóvá. Mivel a szegmentálás és a klasszifikáció nem minden képkockán, hanem csak a felhasználó által kiválasztott pillanatban fut, a rendszer nem nyújt folyamatos, képkockánkénti figyelmeztetést; a polip detektálása tehát az orvos döntésén múlik, mikor nyomja meg a gombot. Emellett a nagyfelbontású bemenet előfeldolgozása (kőzépre vágás, átméretezés) és a CPU–GPU adatátvitel továbbra is jelentős késleltetési tényező maradhat, különösen gyengébb hardveren. A Python interpreter és a GIL hatása egyetlen, igény szerinti inferencia esetén kevésbé kritikus, mint folyamatos feldolgozásnál, de a GUI szál és az inferencia szinkronizálása további optimalizációs lehetőséget jelent a jövőben.

5.3.2 Illeszthetőség a munkafolyamatba

A technikai paramétereken túl vizsgálat tárgyát képezte a rendszer illeszthetősége a tényleges klinikai munkafolyamatba, figyelembe véve az orvosok visszajelzéseit a kezelőfelületről és a vizualizációról. A diszkusszió kitér arra, hogy a rendszer jelenlegi formájában mennyire képes támogatni a döntéshozatalt anélkül, hogy túlzott kognitív terhelést jelentene vagy megzavarná a rutinszerű vizsgálatot. A tapasztalatok azt mutatják, hogy az igény szerinti (on-demand) elemzés jól illeszkedik a gastroenterológiai gyakorlathoz. A kolonoszkópia során az orvos gyanús elváltozás észlelésekor ösztönösen lassít vagy megállítja a képet (freeze) a tüzetesebb vizsgálatához – fotózáskor, dokumentáláskor ugyanúgy megáll a kép. A gombnyomásos logika tehát természetes módon illeszkedik ehhez a meglévő viselkedéshez:

a “Capture & Analyze” funkció ezt a szünetet használja ki, így a 350 ms-os válaszidő nem töri meg a vizsgálat ritmusát, ellenben megerősítést (másodvéleményt) ad a gyanúról. Ez a megközelítés elkerüli a folyamatos valós idejű rendszereknél gyakori fals pozitív villogást, ami elvonhatná a figyelmet a biztonságos eszközvezetéstől.

fejezet 6

Összefoglalás és jövőbeli tervek

Becsült terjedelem: 1-2 oldal

6.1 Az elért eredmények összefoglalása

A dolgozat keretében sikeresen megtervezésre és implementálásra került egy DeepLabV3+ alapú, valós idejű polip szegmentáló keretrendszer, amely hatékonyan integrálja a modern mélytanulási módszereket a klinikai videófeldolgozásba. A létrehozott moduláris Python alkalmazás nemcsak a polipok pontos lokalizációjára képes (83,31% teszt IoU, 90,40% Dice score az SZE tesztalmazon), hanem működő interfészt biztosít a további osztályozási feladatokhoz is – ezzel teljesítve a kitűzött célokat. A rendszer a tesztek során 30 FPS megjelenítési sebességet és 350 ms átlagos válaszidőt ért el integrált grafikus kártyával (ASUS Zenbook 14), bizonyítva, hogy a MobileNetV2 gerinchálózat alkalmas a kompromisszummentes, valós idejű döntéstámogatásra még korlátozottabb erőforrásokkal rendelkező hardveren is.

6.2 A fejlesztés korlátai

A sikerek ellenére a jelenlegi rendszernek vannak technikai korlátai, elsősorban a maximális felbontás és a feldolgozási sebesség közötti kompromisszumkényszer tekintetében. A Python környezet teljesítménybeli határai és a hardveres erőforrások végeessége miatt 4K felbontás esetén még nem garantálható a stabil valós idejű működés, ami további optimalizációt tesz szükségessé. Emellett a jelenlegi dummy osztályozó modul csupán a koncepciót demonstrálja; a klinikai alkalmazhatósághoz elengedhetetlen egy valós patológiai adatokon tanított klasszifikációs hálózat integrálása. A középre vágás 513×513 pixeles bemenetre részletesebb kontextus elvesztését eredményezhet nagyfelbontású forrásnál.

6.3 Jövőbeli fejlesztési irányok

6.3.1 Hardveres optimalizáció

A rendszer teljesítményének drasztikus növelése érdekében a jövőben érdemes megvizsgálni a modell TensorRT-vel történő optimalizálását, vagy a kritikus kódrészek C++ nyelven történő újraírását. Hosszabb távon a rendszer beágyazott eszközökre (pl. NVIDIA Jetson) vagy FPGA alapú hardvergyorsítókra történő portolása tehetné lehetővé a kompakt, hordozható orvosi eszközként való alkalmazást, amely akár közvetlenül az endoszkópos toronyhoz is csatlakoztatható lenne – minimalizálva a külső számítógép függőségét a vizsgálóteremben.

6.3.2 Temporális utófeldolgozás

A detektálási pontosság és a vizuális stabilitás javítása érdekében tervezem temporális (időbeli) utófeldolgozási algoritmusok beépítését. A maszkok egymást követő képkockákon történő simítása vagy átlagolása segíthet kiküszöbölni a detektálás vibrálását, és konziszensebb megjelenítést biztosíthat az orvos számára. Ez különösen a videófolyamaton történő folyamatos elemzés esetén lenne kritikus fejlesztés – jelenleg az igény szerinti módban kevésbé sürgős.

6.3.3 További integráció

A keretrendszer klinikai hasznosíthatóságának növelése érdekében elengedhetetlen a kórházi információs rendszerekkel (HIS/PACS) való integráció megteremtése. Ez lehetővé tenné, hogy a rendszer által készített pillanatfelvételek és leletek automatikusan a beteg elektronikus kórtörténetébe kerüljenek, tovább egyszerűsítve az adminisztrációt és a dokumentációt. A DICOM szabvány támogatása biztosítaná a kompatibilitást a meglévő orvosi képalkotó infrastruktúrával.

Irodalom

- [1] A. M. Leufkens, M. G. van Oijen, F. P. Vleggaar és P. D. Siersema, „Factors influencing the miss rate of polyps in a back-to-back colonoscopy study,” *Endoscopy*, 44. évf., 5. sz., 470–475. old., 2012.
- [2] H. Sung és tsai., „Global Cancer Statistics 2020: GLOBOCAN Estimates of Incidence and Mortality Worldwide for 36 Cancers in 185 Countries,” *CA: A Cancer Journal for Clinicians*, 71. évf., 3. sz., 209–249. old., 2021.
- [3] D. Jha, P. H. Smedsrud, M. A. Riegler, P. Halvorsen, T. de Lange, D. Johansen és tsai., „Kvasir-SEG: A Segmented Polyp Dataset,” *International Conference on Multimedia Modeling*, Springer, 2020, 451–462. old.
- [4] O. Ronneberger, P. Fischer és T. Brox, „U-Net: Convolutional Networks for Biomedical Image Segmentation,” *International Conference on Medical Image Computing and Computer-Assisted Intervention*, Springer, 2015, 234–241. old.
- [5] L.-C. Chen, Y. Zhu, G. Papandreou, F. Schroff és H. Adam, „Encoder-Decoder with Atrous Separable Convolution for Semantic Image Segmentation,” *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018, 801–818. old.
- [6] Z. Yao, L. Wang, Y. Liang, J. Li, J. Zou és Q. Huang, „Lightweight colon polyp segmentation algorithm based on improved DeepLabV3+,” *Journal of Cancer*, 15. évf., 1. sz., 41–53. old., 2024.
- [7] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov és L.-C. Chen, „MobileNetV2: Inverted Residuals and Linear Bottlenecks,” *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, 4510–4520. old.

függelék A

A kolorektális polip szegmentáló modell szoftveres fejlesztésének dokumentációja

A.1 Cél és kontextus

A.1.1 A dokumentáció célja

Jelen melléklet a diplomamunkában bemutatott kolorektális polip szegmentáló rendszer (DeepLabV3+ architektúra) szoftveres megvalósítását és fejlesztési lépéseit dokumentálja. Célja, hogy bemutassa a `colon_short_szd.ipynb` Jupyter Notebook felépítését szoftvermérnöki és adatkutatói (Data Science/ML) szemszögből, biztosítva a kutatás átláthatóságát és a kód reprodukálhatóságát.

A.1.2 Kapcsolat a dolgozattal

Míg a dolgozat 3. fejezete (Módszertan) a modell elméleti hátterét, az architektúra választásának indoklását és a képfeldolgozási elveket tárgyalja, ez a melléklet szigorúan az implementációra fókuszál: hogyan történt a PyTorch osztályok példányosítása, az adathalmazok betöltése, valamint a betanítás és a kiértékelés szoftveres megvalósítása.

A.1.3 Felhasználási scenárió

A szoftver elsődlegesen offline modelltanításra és validációra készült (Jupyter környezetben). A kimenetként kapott betanított modell súlyok (.pth fájl) a későbbiekben felhasználhatók önálló inferencia szkriptekben (pl. `live_capture_app.py`), valós idejű vagy rögzített endoszkópos videók/képek feldolgozására.

A.2 Fejlesztési környezet és függőségek

A projekt fejlesztése és futtatása lokális, illetve felhőalapú GPU környezetben történt a megfelelő teljesítmény elérése érdekében.

A.2.1 Hardverkörnyezet

- **Számítási egység:** Nvidia GPU (pl. RTX széria vagy Google Colab T4) a gyorsított tenzorműveletekhez. Egy epoch tanítási ideje a batch mérettől és a GPU teljesítményétől függően változik.
- **Rendszermemória:** Minimum 16 GB RAM javasolt a képek betöltéséhez és az adat-augmentációhoz.

A.2.2 Szoftverkörnyezet és verziók

A fejlesztés Python nyelven, Jupyter Notebook / Cursor / VSCode környezetben zajlott. A legkritikusabb függőségek a következők:

- **Operációs rendszer:** Windows / Linux (WSL)
- **Python verzió:** 3.10+
- **PyTorch:** torch és torchvision (CUDA támogatással)
- **Egyéb elengedhetetlen könyvtárak:** numpy, matplotlib, Pillow, scikit-learn

A.2.3 Környezet felállítása

A projekt egy dedikált virtuális környezetben (venv) futtatható. A telepítés a következőképpen történik:

```
python -m venv venv
# Windows esetén:
venv\Scripts\activate
# Linux/Mac esetén:
source venv/bin/activate

pip install -r requirements.txt
```

A.3 Rendszerarchitektúra magas szinten

A rendszer adatfolyama moduláris felépítésű, követve a gépi tanulási projektek klasszikus életciklusát:

1. **Adatbetöltés és -felosztás:** Képek és maszkok szinkronizált betöltése, majd felosztása (train/val/test).
2. **Adat- és maszkolás előfeldolgozás:** Képek transzformációja (augmentáció) Pytorch Dataset osztályok segítségével.
3. **Modell definíció:** A torchvision könyvtár DeepLabV3+ modelljének inicializálása MobileNetV2 encoderrel.
4. **Tanítási ciklus:** Mini-batch alapú iteráció, veszteségfüggvény számítása, backpropagation.
5. **Kiértékelés és mentés:** Validációs metrikák (IoU, Dice) számítása, a legjobb modell-állapot (checkpoint) automatikus mentése.

(Az adatfolyam vizuális ábrázolását a dolgozat vonatkozó fejezete tartalmazza.)

A.4 Adatkezelés és előfeldolgozás

A.4.1 Címtárszerkezet

A kód az alábbi dedikált mappastruktúrát várja az adatbázisokhoz (Kvasir-SEG és a SZE Dataset beolvasztása után):

```
projekt_gyökér/
├── images/
│   ├── polyp_1.jpg
│   ├── polyp_2.jpg
│   └── ...
├── masks/
│   ├── polyp_1.jpg (vagy .png)
│   ├── polyp_2.jpg
│   └── ...
└── checkpoints/
```

A betöltő szkript párosítja az azonos nevű képeket és maszkokat.

A.4.2 PyTorch Dataset és Dataloader

A képek beolvasása egy egyedi, `torch.utils.data.Dataset` osztályból származtatott implementáción keresztül történik.

- A **DataLoader** felel a képek kötegeléséért (batching), jellemzően `batch_size=4` értékkel, a GPU memória korlátai miatt.

- A betanító DataLoader esetében a `shuffle=True` beállítás biztosítja az adatok véletlenszerűsítését.

A.4.3 Transzformációk

Az augmentáció a `torchvision.transforms` segítségével valósul meg a betanítási fázisban:

- **RandomScale / Resize:** A képek méretének véletlenszerű változtatása.
- **RandomCrop (513x513):** A hálózat rögzített bemeneti méretének biztosítása.
- **RandomHorizontalFlip / RandomRotation:** Forgatások és tükrözések a modell robustusságának növelésére.
- **Normalize:** Az ImageNet statisztikáinak (mean, std) megfelelő normalizáció, amely kritikus a transzfer tanulás (pretrained súlyok) sikerességéhez.

A.5 Modellkomponens

A rendszer központi eleme a szegmentáló hálózat, amelynek inicializálása a `torchvision` modulból történik.

A.5.1 Inicializálás és architektúra

A kód a `torchvision.models.segmentation.deeplabv3_mobilenet_v3_large` (vagy a korábbi V2-es alapú) implementációt hívja meg.

- **Pretrained súlyok:** `weights='DEFAULT'` (COCO/VOC alapú előtanítás).
- **Kimeneti réteg (Classifier):** Mivel a feladat bináris szegmentálás (háttér vs. polip), az utolsó konvolúciós réteget a kód lecseréli, hogy pontosan `num_classes=2` kimeneti csatornája legyen.

A.5.2 Veszteségfüggvény és Optimalizáló

- **Veszteségfüggvény (Loss):** `nn.CrossEntropyLoss()`, amely pixel-szintű többszörös (itt 2 osztályos) klasszifikációra optimalizál.
- **Optimalizáló algoritmus:** `torch.optim.Adam` inicializálva `lr=0.001` ($1e-3$) tanulási rátával.
- **LR Scheduler:** `ReduceLR0nPlateau` osztály, amely automatikusan csökkenti a tanulási rátát (pl. `factor=0.1`), ha a validációs IoU nem javul meghatározott számú (patience) epoch után.

A.6 Tanítási és validációs folyamat

A tanítás logikája egy kifejezett tréning ciklusba van szervezve.

A.6.1 Tanító ciklus

Minden epochban a kód végigiterál a betanító adathalmazon:

1. Az adatok GPU-ra másolása (`.to(device)`).
2. Előrehaladás (forward pass): `outputs = model(images)`.
3. Veszteség számítása a maszkokhoz viszonyítva.
4. Visszaterjesztés (backward pass): `loss.backward()`.
5. Súlyok frissítése: `optimizer.step()`, és a gradiensek nullázása: `optimizer.zero_grad()`.

A.6.2 Validáció és korai megállás

Minden tanítási epoch végén a hálózat `model.eval()` módba kapcsol, és lefut a validációs adathalmazon.

- **Metrikák:** Dice együttható és Intersection over Union (IoU) számítása történik pixel-szinten.
- **Early Stopping:** A kód figyeli a validációs IoU-t. Ha bizonyos számú epochon keresztül (pl. `patience=7`) nincs javulás, a tanítás korábban leáll, megelőzve a túltanulást (overfitting).
- **Naplózás (Logging):** A metrikák (loss, lr, IoU) historikus adatai memóriában (szótárként) tárolódnak, majd a futás végén kirajzolódnak a matplotlib segítségével.

A.7 Eredmények reprodukálhatósága

A gépi tanulási modellek megbízhatóságának alapja a reprodukálhatóság. Ennek érdekében a kód az alábbi elveket követi:

A.7.1 Seed beállítások

A futtatás elején egy központi függvény beállítja a determinisztikus generátorokat:

```
import torch
import numpy as np
import random
```

```
def set_seed(seed=42):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    # CUDNN backend determinizmus beállítása (opcionális, de ajánlott)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False

set_seed(42)
```

A.7.2 Hiperparaméter konfiguráció

A végső betanított modellhez tartozó főbb hiperparaméterek (melyekkel az eredmények reprodukálhatók):

- **Batch size:** 4
- **Max Epochs:** 50 (Early stoppinggal megállhat hamarabb is, pl. 15-20 körül)
- **Kezdeti LR:** 0.001
- **Optimizer:** Adam
- **Kép bemeneti méret:** 513x513

A.8 Kimenetek és használat

A.8.1 Checkpoint kezelés

A kód csak a legjobb modellt (best IoU) menti le a lemeze: checkpoints/best_model_epoch_{epoch}.p Ez a fájl tartalmazza a modell súlyait (state_dict), az optimalizáló állapotát és a tanulási metrikákat.

A.8.2 Predikciós pipeline kód minta

A betanított modell gyakorlati felhasználása (inference) az alábbi minimális kódrészlettel végezhető el egy új képre:

```
import torch
from torchvision.models.segmentation import deeplabv3_mobilenet_v3_large
from PIL import Image
```

```

import torchvision.transforms as T

# 1. Modell inicializálása és súlyok betöltése
model = deeplabv3_mobilenet_v3_large(num_classes=2)
checkpoint = torch.load('checkpoints/best_model_epoch_12.pth', map_location='cpu')
model.load_state_dict(checkpoint['model_state_dict'])
model.eval()

# 2. Kép betöltése és előfeldolgozása (ugyanaz a normalizáció, mint tanításnál)
img = Image.open('test_image.jpg').convert('RGB')
transform = T.Compose([
    T.Resize((513, 513)),
    T.ToTensor(),
    T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])
input_tensor = transform(img).unsqueeze(0) # Batch dimenzió hozzáadása

# 3. Predikció (Inferencia)
with torch.no_grad():
    output = model(input_tensor)['out']
    # Osztály valószínűségek és argmax (0: háttér, 1: polip)
    prediction = torch.argmax(output.squeeze(), dim=0).byte().cpu().numpy()

# A 'prediction' változó most egy 513x513-as bináris maszkot tartalmaz.

```

A.9 Ismert limitációk és jövőbeli bővítés

Bár a jelenlegi szoftverarchitektúra stabilan működik, szoftvermérnöki szempontból az alábbi korlátokkal és fejlesztési lehetőségekkel érdemes számolni:

- **Korlátok:** A `batch_size=4` relatív kicsi, amely a GPU memória (VRAM) korlátaiból fakad. Ez némileg zajosabb gradiens-frissítést eredményez. Az adathalmaz mérete (bár kiterjesztett) még mindig hagy teret a túltanulásnak, amit az early stopping hivatott tompítani.
- **Továbbfejlesztés (Architektúra szinten):** Konfigurációs fájlok (pl. `config.yaml`) bevezetése a notebookba keménykódolt változók (hardcoded variables) helyett.
- **Továbbfejlesztés (Inferencia):** A valós idejű endoszkópos felhasználáshoz a betanított PyTorch modell ONNX vagy TensorRT formátumba történő konvertálása javasolt a másodpercenkénti képkockaszám (FPS) drasztikus növelése érdekében.

függelék B

A valós idejű szegmentáló alkalmazás szoftveres dokumentációja

B.1 Cél és kontextus

B.1.1 A dokumentáció célja

Jelen melléklet a diplomamunkában bemutatott valós idejű polip szegmentáló alkalmazás (`live_capture_app.py`) szoftveres megvalósítását dokumentálja. Célja, hogy bemutassa az alkalmazás felépítését szoftvermérnöki szemszögből: a videófolyam kezelését, a szegmentáló modell integrációját és a felhasználói felület implementációját.

B.1.2 Kapcsolat a dolgozattal

Míg a dolgozat 4. fejezete (Szoftveralkalmazás) a rendszer követelményeit, az adatfolyam diagramot és a felhasználói élményt tárgyalja, ez a melléklet szigorúan az implementációra fókuszál: hogyan valósul meg a `VideoCaptureThread`, a `MedicalAnalyzer` és az `App` osztály, valamint a közöttük lévő adatáramlás.

B.1.3 Felhasználási scenárió

Az alkalmazás élő videóforrásból (HDMI capture kártya, webkamera, videófájl) vagy szimulációs módban (képmappa) folyamatosan megjeleníti a képet, és a felhasználó gombnyomására (Space) vagy „Capture & Analyze” gombra igény szerint elvégzi a polip szegmentálást és a demonstrációs klasszifikációt. A kimenet három panelen jelenik meg: élő kép, hálózati bemenet, szegmentált eredmény.

B.2 Fejlesztési környezet és függőségek

B.2.1 Hardverkörnyezet

- **Bemeneti források:** HDMI capture kártya, USB webkamera, vagy helyi videófájl. Szimulációs módban: képmappa (images/).
- **Számítási egység:** CPU vagy GPU (CUDA); a modell automatikusan detektálja a rendelkezésre álló eszközt (`torch.device('cuda' if torch.cuda.is_available() else 'cpu')`).
- **Tesztelt konfiguráció:** ASUS Zenbook 14 (AMD Ryzen 7, integrált grafikus) – a 5. fejezetben bemutatott teljesítménymérések alapján.

B.2.2 Szoftverkörnyezet és verziók

- **Python:** 3.x
- **Kritikus függőségek:** opencv-python (cv2), torch, torchvision, PIL (Pillow), numpy, tkinter (GUI, általában Python része)
- **Projekt-specifikus:** `network.modeling(deeplabv3plus_mobilenet)`, `utils.ext_transforms` (ExtResize, ExtCenterCrop, ExtToTensor, ExtNormalize)
- **Opcionális:** pygrabber (DirectShow kameralista Windows-on; hiányában egyszerű indexelés)

B.2.3 Környezet felállítása és futtatás

- A projekt gyökérkönyvtárából: `python live_capture_app.py`
- A checkpoint elérési útja: `checkpoints/best_model_epoch_38.pth` (konfigurálható a fájl elején)

B.3 Rendszerarchitektúra magas szinten

A rendszer három fő modulból áll, amelyek a dolgozat 4. fejezetének adatfolyam diagramjához igazodnak:

1. **VideoCaptureThread:** Dedikált szálon folyamatosan olvassa a videóforrást (kamera, videófájl) vagy szimulációs módban véletlenszerű képeket tölt be. A legfrissebb képkocka szálbiztos lock-kal védett.

2. **MedicalAnalyzer:** Betölti a DeepLabV3+ modellt, előfeldolgozza a képet (513×513, ImageNet normalizáció), inferenciát futtat, vizualizálja a maszkot és naplózza a teljesítményt (CSV).
3. **App (GUI):** Tkinter alapú hárompaneles felület, kezeli a forrásválasztást, a „Capture & Analyze” műveletet és a Space billentyű kötését.

B.4 Modulok részletesen

B.4.1 VideoCaptureThread

- **Felelősség:** Videófolyam vagy szimulációs képek folyamatos betöltése dedikált daemon szálon.
- **Bemenetek:** `src` (kamera index int vagy videófájl útvonal string), `simulation_mode`, `image_folder`.
- **Szimulációs mód:** `images/` mappa, szűrés `allowed_ids` lista alapján (pl. PI009, PI015, ...). 2 másodpercenként véletlenszerű kép váltás.
- **Videófájl mód:** `set_position(percent)` – pozíció ugrás a sávval (slider). Hurok a fájl végén.
- **Kimenet:** `read()` – (ret, frame) tuple, szálbiztos.

B.4.2 MedicalAnalyzer

- **Felelősség:** Modell betöltése, előfeldolgozás, inferencia, vizualizáció, teljesítménynaplózás.
- **Modell inicializálás:** `deeplabv3plus_mobilenet(num_classes=2, output_stride=16)`. Checkpoint kulcsok: `model_state`, `model_state_dict`, `state_dict` vagy gyökér.
- **Előfeldolgozás:** `ExtResize(513) → ExtCenterCrop(513) → ExtToTensor → ExtNormalize` (ImageNet mean/std) – összhangban a notebook validációs transzformációival.
- **Inferencia:** `model(input_tensor), output.max(1)[1] → bináris maszk (513×513)`.
- **Vizualizáció:** Polip maszk alapján az eredeti kép tartalma megjelenik, háttér fekete.
- **Klasszifikáció:** Jelenleg `dummy_classify()` – véletlenszerű demonstrációs eredmény (Adenoma/Hyperplastic/Serrated). Interfész készen áll valódi modell cseréjére.
- **Naplózás:** `performance_log.csv` – `Timestamp`, `Total_Time_ms`, `Preprocess_ms`, `Inference_ms`, `Postprocess_ms`.

B.4.3 App (GUI)

- **Layout:** Grid: felső sor (forrásválasztó, gombok, Live Analysis checkbox, videó slider), középső sor (3 panel: Live Feed, Network Input, Polyp Segmentation), alsó sor (Capture & Analyze gomb, státusz).
- **Forrásválasztás:** Combobox – kameralista (`list_available_cameras()`), videó-fájlok a projekt mappájában. „Start Live Input” / „Simulation (Images)” gombok.
- **Billentyűkötés:** Space → `capture_and_analyze()`.
- **Live Analysis:** Checkbox bekapcsolásakor minden GUI frissítési ciklusban (33 ms) lefut az elemzés – nagy terhelés, főleg demonstrációs célra.
- **Panel méretek:** Megjelenítés 400 px magasságúra skálázva.

B.5 Konfiguráció és fájlstruktúra

B.5.1 Konfigurációs változók

- `PERFORMANCE_LOG_FILE` – CSV napló neve
- `CHECKPOINT_PATH` – modell súlyok útvonala
- `NUM_CLASSES`, `OUTPUT_STRIDE`, `INPUT_SIZE` (513)
- `DEVICE` – automatikus CUDA/CPU választás

B.5.2 Várt fájlstruktúra

```
projekt_gyökér/
├── live_capture_app.py
├── network/
│   └── modeling.py           # deeplabv3plus_mobilenet
├── utils/
│   └── ext_transforms.py     # ExtResize, ExtCenterCrop, stb.
├── checkpoints/
│   └── best_model_epoch_38.pth
├── images/                   # Szimulációs mód (opcionális)
│   └── *.jpg, *.png
└── performance_log.csv      # Generált futás közben
```

B.6 Adatfolyam és szálkezelés

B.6.1 Szálmodell

- **Capture szál:** Daemon thread, folyamatosan olvassa a forrást, lock-kal védi a frame változót.
- **GUI szál (main):** `root.after(33, update_live_feed)` – 30 FPS megjelenítés. A `capture_and_analyze` szinkron módon fut a főszálon (blokkoló).
- **Konfliktusok elkerülése:** `VideoCaptureThread.set_position` lock-kal védett, hogy a seek ne ütközzön az olvasással.

B.6.2 Elemzési folyamat (Capture & Analyze)

1. `video_thread.read()` – pillanatnyi képkocka
2. `analyzer.analyze(frame)` – preprocess → inference → postprocess → vizualizáció
3. Panel frissítés (snapshot, result)
4. Státusz szöveg: klasszifikációs eredmény + feldolgozási idő

B.7 Ismert limitációk és jövőbeli bővítés

- **Klasszifikátor:** Jelenleg dummy; valódi patológiai modell integrálása szükséges a klinikai használathoz.
- **Teljesítmény:** 350 ms válaszidő CPU-n (Zenbook); GPU-val vagy TensorRT optimalizálással javítható.
- **4K / nagy felbontás:** Középre vágás 513×513-re – részletesebb kontextus elveszhet.
- **Platform:** A pygrabber és a DirectShow kameralista Windows-specifikus; Linux/Mac más megoldást igényel.

B.8 Reprodukálhatóság és futtatási lépések

- **Checkpoint:** A notebookban tanított `best_model_epoch_38.pth` (vagy aktuális legjobb) másolása a checkpoints/ mappába.
- **Futtatás:** `python live_capture_app.py` a projekt gyökeréből.
- **Szimulációs teszt:** `images/` mappa létrehozása, releváns páciens ID-jú képek másolása (pl. PI009, PI015, ...).